CO1035: Game Concepts
Lab 1 – Introduction to TL-Engine and Game Programming

1. Introduction

- This is your first lab on game development with TL-Engine. This lab worksheet will help you write your first TL-Engine program.
- This lab sheet involves a lot of reading as concepts and tools are introduced. Later lab sheets will tend to be shorter, with more work for you to do!
- Finally, you must name the projects you create in each lab with the names suggested in the lab sheet, and save them in a secure and accessible location (e.g., your UCLan-provided cloud storage or GitHub). This is important because references will be made to projects from previous labs as we progress through the module.

2. Starting a new TL-Engine Project

- Start by creating a new TL-Engine project as described in the installation file.
- This will open Visual Studio with a TL-Engine project ready to run.

3. Using C++ Syntax (Reading if Lab 1A is completed)

a. **C++** is a programming language that gives programmers high-level control over system resources and memory. It is one of the world's most popular programming languages and the language used to build most big console and PC games.

b. C++ programming will be taught to you as part of another module. However, this module will use the TL-Engine to introduce you to the features and concepts of C++ that are standard in the game development industry.

c. C++ has different variable types for storing different types of data (**int** for storing integers (whole numbers), **bool** for storing Boolean values, **string** for storing text). The data type of a variable must be stated when declaring it to specify the type of data that the variable can store. Examples of variable declaration in C++ are shown below:

```
int length; // an integer variable named length is declared for storing whole numbers
string name; // a string variable named name is declared for storing text data
float speed; // a float variable named speed is declared for storing floating
point - decimal numbers
bool check; // a boolean variable named check that can only take the values
'true' and 'false'
```

d. You can assign a value to a variable. Assignment is done using the equals symbol and can be done when or after declaring a variable.

```
int number = 10; // number is assigned a value of 10 when declared
bool check; //check is first declared
check = true; // then check is assigned a value of true
```

Note: Assignment is not the same as 'equals'!

e. The contents of a variable can be changed as shown in the examples below:

```
int i = 4;
i++; // i= 5 as the ++ operator increments the value of a variable by 1
bool check = true;
check = false; // the value of check is changed to false
string name;
name = "Alex"; // the value of name is Alex
name = ""; // the value of name is changed to an empty string
```

f. While using the TL-Engine you will be using standard C++ variable types as well as custom variable types. An example of a variable declaration using a custom type is shown below:

```
I3DEngine* myEngine; // TL-Engine specific type for storing instances of a 3D engine
```

Note: The type 'I3DEngine*' is not a C++ type; it is specific to the TL-Engine. Also note that it is being used with a 'pointer' – the symbol '*'. You will be introduced to both custom types and pointers later. For now, just remember that 'I3DEngine*' is a type just like an int or a float, but specific to the TL-Engine.

g. The TL-Engine has many 'functions' that you can use. A function is a block of program code that performs a particular calculation or task. You can use a built-in C++ function in your own code when you want to perform a calculation or task – it saves you from writing the code yourself. There are some C++ functions that you may have seen already:

```
float squareRoot = sqrt(16); // C++ function to calculate the square root of a number
system("pause"); //a function that issues a special system command to the computer
```

h. In the 'sqrt' example, notice how the number 16 is 'passed' to the function (in brackets), e.g., given as input to the function. Also notice that the 'sqrt' function calculates a result – and this result is put into the variable 'SquareRoot'.

i. In the second example, the string 'pause' is passed to the 'System' function, but there is no result – this function simply performs a task (waits for a key press).

j. You will see functions like this in the TL-Engine. Some are passed values (sometimes several values), and some are not. Also, some return results, although again, some don't. The use of each function will be explained to you in the worksheets, with examples given.

k. Finally, most functions in the TL-Engine must be used through a variable:

```
I3DEngine* myEngine; // I3Dengine is a TL-Engine specific variable type
myEngine->DrawScene(); // DrawScene is a function ("method") of myEngine
```

In the above code, a variable named 'myEngine' is declared, then 'DrawScene()' is used on the variable (notice the use of the '->' symbol to indicate this usage). In plain English, the two lines of code state: "create a 3D engine and call it myEngine, then use myEngine to draw a scene". Functions like DrawScene() are called methods.

4. The TL-Engine Template Code

a. Open your project created in Visual Studio and open the file with the extension '.cpp' using Visual Studio's "Solution Explorer" to the right of the screen. You can see that the TL-Engine wizard provides you with a template source code; **don't delete it – it is useful!**

b. The template follows a structure that is common to game programs known as the "game code structure". The structure provides sections to set up, run, and terminate games properly. These sections are ***initialization***, ***the game loop,*** and ***termination***; they also appear in the code snippet below.

```
#include < game engine libraries >
Start game engine
//initialisation
Load game objects from disk
Set initial positions for game objects
//game loop
while (game is playing)
{
Draw the game scene on screen
Move/Animate game objects
}
//termination
Release resources
Stop game engine
End program
```

c. Now let us go over our template code. The first line contains a comment, followed by the inclusion of the TL-Engine file, which contains all the information needed to use the engine. Following the include statement is a "using namespace" statement: all of the TL-Engine is 'hidden' inside a 'namespace' so it does not get mixed up with standard C++ or other libraries. This statement exposes the namespace so we can use its contents easily. These opening lines must begin all programs using the TL-Engine; you do not need to understand their details, just make sure they are there.

```
// Lab 1 - Introduction.cpp: A program using the TL-Engine
#include <TL-Engine.h> // TL-Engine include file and namespace
using namespace tle;
```

d. In the next lines, we see the 'main' function; it is where the execution of all C++ programs begins.

```
int main()
{
	…
}
```

e. The main function begins with the initialization section. Everything that is needed to get the game going is set up in this section. This includes initializing variables and loading resources (e.g., models and sounds).

```
// Create a 3D engine (using TLX engine here) and open a window for it
I3DEngine* myEngine = New3DEngine( kTLX );

myEngine->StartWindowed();

// Add default folder for meshes and other media
myEngine->AddMediaFolder("C:\\ProgramData\\TL-Engine\\Media");

/**** Set up your scene here ****/
```

In the code above, a variable called **'myEngine'** is declared and its type is an 'I3DEngine*'. All you need to know now is that the variable **'myEngine'** will be your interface to the TL-Engine, meaning you will use the TL-Engine's features 'through' this variable.

After declaring the '**myEngine**' variable, it is initialised by using a function/method: **'New3DEngine(kTLX)**''. In plain English, that means "I want to use a 3D engine of the TLX variety, and I will call it **myEngine**".

The next line is the first use of this new engine we have prepared.
See how we use "**myEngine->StartWindowed()**" to say, "I want **myEngine** to start up in a window".

The next line tells the engine to look in the folder provided (the folder chosen when TL-Engine was installed) for any files (e.g. 3D objects) that you load later on. Any resources you want to load to your game should be loaded after this line of code.

f. Next comes the game loop section of the template code, which executes in a loop until told to stop.

```
// The main game loop, repeat until engine is stopped
while (myEngine->IsRunning())
{
	// Draw the scene
	myEngine->DrawScene();
```

```
        /**** Update your scene each frame here ****/
}
```

Currently, the only code inside this game loop is code that tells the 3D engine declared in the initialization stage (**myEngine**) to draw the scene, and an if-statement that tells the engine that if the **Key_Escape** is pressed, then the engine should terminate the program. Further code can be added to the game loop to check for player input and update game objects in each iteration. The game loop's ability to perform several iterations every second and display updated *frames* creates the animation seen when playing games.

g. The final section, the termination section, cleans up after the game and releases all the memory resources used by the game. It contains a single line of code that deletes the 3D engine that you created at the beginning:

```
// Delete the 3D engine now we are finished with it
myEngine->Delete();
```

If we don't do this, then the resources used by the 3D engine will remain in the computer's memory after the program is finished, but no other program will be able to access them. This is called a memory leak - it will affect the performance of the computer, so this line is important. Now that we have some understanding of what the code template does, let us add code to make things more interesting.

5. Adding Your Own Code

a. In this section, we will add code to the template code to declare new variables, load a mesh, create a model, and a camera to display our scene.
b. Start by adding the code below in the initialization section. Place it below:

```
/**** Set up your scene here ****/
IMesh* cubeMesh;
IModel* cube;
```

The above code declares two variables, a mesh variable and a model variable. Note how we use the same pointer syntax (*) as before when declaring a 3D engine variable; these variables will be used in the same way as **'myEngine'**.

A **mesh** is a 3D object generated by an artist and saved in a file. A model is a single copy of a mesh in our scene. At first, this might seem similar, but remember that **a single mesh can be used to create several models in your scene** (e.g., we may have a single tree mesh, but make a forest in our game using 20 models created from this mesh). Consider a mesh as a **'blueprint'** for creating **models**.

c. Next, type the code below:

```
cubeMesh = myEngine->LoadMesh("Cube.x");
cube = cubeMesh->CreateModel();
```

The first line here uses the 3D engine (myEngine) to load a file and put it into our declared mesh variable (cubeMesh). Notice how we need to go 'through' the variable 'myEngine' again using "->". Think about this as asking myEngine to perform a service for us (loading a mesh) - the 3D engine must do this job, as it needs to keep track of all new meshes. So in English, the code is saying: "Use myEngine to load a mesh called 'cube.x' and put it in the variable 'cubeMesh'".

In the second line, the mesh is used as a blueprint to create a model. Note that the syntax is similar to what we used to load the mesh using the 3D engine, but this time the mesh is used to create a model. In English, the code is saying: "Use the cubeMesh blueprint to create a model in the scene and put it in the variable 'cube.'".

d. The last piece of code we need to add to the initialisation section is the code below:

```
ICamera* myCamera;
myCamera = myEngine->CreateCamera(kFPS);
```

The first line declares a variable named **'myCamera'** that will be used to hold a camera. The second line of code uses a syntax similar to what we have used when loading a mesh and creating a model. In English, the code is saying: "Use **myEngine** to create a camera of type FPS and put it in the variable **myCamera**". FPS is a "first-person shooter" camera that can be moved around with the mouse and cursor keys.

e. We are now done with the initialization section, and the game loop section already has code that draws the scene when the game is executed. Therefore, when you run the project now, the 3D engine draws the scene using the viewpoint of our camera - the scene will be drawn into the window we opened earlier (in the initialization section). We asked for a controllable FPS camera, so the image drawn will update and change as we move the camera around using the keyboard or mouse.

f. Run the program (press the play icon or F5). You should now see a model, and you can 'fly' around using the mouse and cursor keys.

g. When you run your project, your FPS camera may be moving too fast; you can add a delay loop to slow it down. Add the code that follows inside the game loop immediately after `/**** Update your scene each frame here ****/` and run your project again.

```
for(int delay=0; delay < 1000000; delay++)
{
        /* empty body */
}
```

h. Any changes or updates on game objects while the game is executing need to be performed in the game loop.

6. Practice Exercises

The sections that follow are practice exercises on loading, positioning, and movement; please attempt all these exercises.

Loading

a. Here is a list of mesh filenames that can be loaded with the **'LoadMesh'** function that we used in 5.3 (section 5, task 3). Those filenames are located in the media folder of TL-Engine. There are more there. To explore the different meshes, follow the path that is located in your code. This path demonstrates where the TL-Engine media folder is installed.
Filenames: "Cube.x", "Torus.x", "Sphere.x", "Grid.x","TwoPence.x"

b. Our existing code can be updated to display a sphere instead of a cube by changing the name of the loaded mesh in 4.3 from **'Cube.x'** to **'Sphere.x'**.

c. Update the code to display a torus using the **'Torus.x'** mesh and run the program to make sure you have achieved the desired result.

d. If we want to load more than one mesh at a time, we need to declare additional **IMesh*** variable(s) to put the new mesh in. Now display both the **'Cube.x'** and **'Torus.x'** in your program. Remember, you will need to add new mesh and model variables (with sensible new names).

Positioning

a. All the models we have created so far have been created in the same position in the scene. We can choose to create our models at different positions by using "Cartesian coordinates" to specify exactly where in the scene we want a model to be created.

b. It is easier to understand Cartesian coordinates by loading an example model.

c. Save your project, then create and open a new TL-Engine project.

d. Add code to load and display a grid using the "**Grid.x**" mesh. You can copy some of your code from the previous project.

e. When you run your project, a grid will be displayed. The grid has lines that are marked with numbers, and each line represents a Cartesian coordinate axis. The X axis runs horizontally from left (negative) to right (positive) of the grid; the Y axis runs vertically from down (negative) to up (positive), and the Z axis goes from the screen (negative) into the scene (positive); these are the three axes (plural form of axis, pronounced 'ack-sees') of the Cartesian coordinate system. At the center of the model is the origin; it is marked with 0 because at this point the values of X, Y, and Z are all 0.

f. Now add code to your project to display a cube so that your project displays both the grid and the cube. Run your project.

g. You will notice that the cube is created at the center of the grid, where the 0 marking is, i.e., where x = 0, y = 0, and z = 0. This is because whenever the CreateModel method is used to create a model without specifying the initial position of the model, it is created at the origin of the scene.

h. Before you change the position of your cube in your code, play around with the values of x, y, and z in the interactive scene below to understand how the values of Cartesian coordinates can be used to determine unique positions.

i. Now update your code by passing three values to set the initial position of the cube as shown below:

```
cube = cubeMesh->CreateModel(10, 0, 20);
```

This positions the model using 'Cartesian coordinates', with **x = 10**, **y = 0,** and **z = 20**. Run the project and see the new position of your cube. Is the cube where you expected it to be? Set the x, y, and z values in the interactive scene to the values used in your code to verify your result.

j. Experiment with a few different x and z positions for the cube. Then try altering the y position. Keep all the values less than 50. Can you tell the length of each of the cube's sides?

k. From the scenes your program displayed and the scene snapshots shown above, can you tell the length of each of the cube's sides?

l. You should have noticed that the length of the sides of the cube is 10. This is clearer when you position the cube so that it sits on the floor of the grid and to the right of the origin using the code below:

```
cube = cubeMesh->CreateModel(5, 5, -5);
```

m. Add another cube to your program by declaring a new **IModel*** variable and putting a cube mesh model into it. Remember that you do not have to declare a new IMesh* variable and load a new cube mesh whenever you want to create an additional cube; you can create as many cube models as you want with the cube mesh you have loaded already (as long as you have the **IModel*** variables to put them into).

n. See if you can stack the two cubes you have on top of each other.

o. Add and display a sphere in your program.

p. Now stack two cubes on the sphere.

q. Save your project at this point by selecting 'File->Save All'

Rotation and Movement

a. Create a new TL-Engine project and name it **Lab1B_Movement_Project**. Open the project.

b. Write code to load and display a grid and a cube (copy some of your code from the previous project).

c. In the game loop of our program, we can add code to update the scene, e.g., to move or rotate our models. If we move a model by a small amount inside the game loop, then it will move continuously as the loop repeats; the same will happen with rotation.

d. Add this line inside the game loop:

```
cube->RotateY( 0.05 );
```

This will make the cube rotate by a small amount each time the loop repeats. This effectively spins the cube.

e. Now change the sign of the parameter passed to the rotation method in your code, i.e., change it from 0.05 to -0.05. Run your project and note the difference.

f. Hopefully, you noticed that the cube spins around its y-axis as indicated by the name **'RotateY'**, and the direction of its spin is determined by the sign of the parameter passed (i.e., whether it's negative or positive). Can you guess what will happen if the cube is rotated about its x-axis? What if it is rotated about its z-axis? What if the rotation method is placed outside the game loop?

g. Here is a list of some of the movement and rotation functions available for models. Note that the word 'void' on the left means that these functions don't calculate a value; they just perform a task. The word 'float' in the brackets means you must pass a float value to these functions (the amount to move or rotate). The examples written as comments on the right show how these functions should appear in your code:

```
void RotateX( float ); // e.g. cube->RotateX( 0.05 );
void RotateY( float ); // e.g. sphere->RotateY( -0.1 );
void RotateZ( float ); // e.g. torus->RotateZ( 0.3 );
void MoveX( float ); // e.g. torus->MoveX( 0.1 );
void MoveY( float ); // e.g. cube->MoveY( -0.2 );
void MoveZ( float ); // e.g. cube->MoveZ( 0.4 );
```

h. Now write the appropriate code to rotate the cube around the other two axes.

i. Try to use the MoveX function to move the cube to the left.

j. Now change the code so that the cube moves to the right.

k. Save your project

Further Positioning

a. There are more functions available for positioning, e.g., for setting the position of an object, and for getting the position of an object.

```
// Set functions that allow to set the XYZ coordinates of a model to a given value
void SetX( float ); // e.g. cube1->SetX( -20 );
void SetY( float ); // e.g. cube2->SetY( 10 );
void SetZ( float ); // e.g. cube2->SetZ( 7.5 );


// Set all axes at once (as CreateModel)
```

```
void SetPosition (float X, float Y, float Z); //e.g. myModel->SetPosition(20,0,10);

// The Get functions return the current X, Y or Z position of a model
float GetX(); // e.g. xpos = sphere->GetX();
float GetY(); // e.g. if (sphere->GetY() > 10)
float GetZ(); // e.g. zpos = sphere->GetZ();
```

Note that the 'Get' functions calculate a float value (indicated by the word ‘float’ on the left), look carefully at the example usage.

b. Using the appropriate method from those provided above, update your code so that your camera is positioned at (0, 0, -50).

7. Moving Forward: Positioning Exercises

In this exercise, you will be creating two structures, "The Pyramid" and "The Giant". The purpose of this exercise is to provide you with practice on creating and positioning models using Cartesian coordinates.

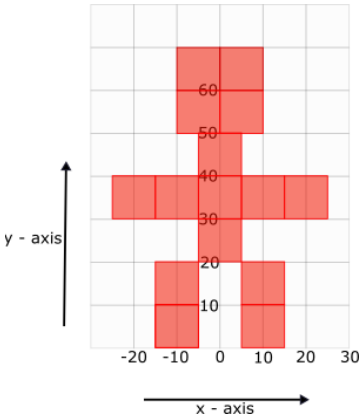
The Giant

a. Create a new TL-Engine project and name it **Lab1B_Giant_Project**.

b. Add an FPS camera to your project.

c. Create an **IMesh*** variable and load a cube mesh ("**Cube.x**") into it.

d. In the lab, we measured the length of the sides of the cube model using a grid. We found that it has sides of length 10. With that measurement, The Giant can be represented as a collection of cubes in a 2D grid (showing x and y axes).



e. You are to use the image provided above as a blueprint for creating The Giant.

f. From the image, you should be able to see that you need 15 cube models to create the structure. Therefore, declare 15 **IModel*** variables, name them appropriately (or you will, without doubt, confuse yourself), and create a model for each using the cube mesh.

g. The Giant should stand at the origin; therefore, all the cubes that make it up should have 0 as their z-coordinate position.

h. However, each cube has a different x and y coordinate, and by looking at the 2D image of The Giant, you can easily identify the x and y coordinates for each cube by drawing a line from the center of each cube to the x and y axes.

i. For example, the bottom left block is positioned at x = -10 and y = 5; therefore, it should be created using **CreateModel(-10, 5, 0);** and the bottom right block is positioned at x = 10 and y = 5; therefore is should be created using **CreateModel(10, 5, 0);**

j. Now use the blueprint to figure out the position of all the cubes that make up "The Giant", and write the code in your program to position them.

The Pyramid

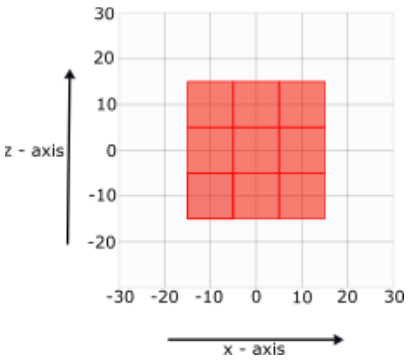
a. Create a new TL-Engine project and name it **Lab1B_Pyramid_Project**.

b. Add an **FPS** camera to your project.

c. Load a cube mesh (**"Cube.x"**) and a sphere mesh (**"Sphere.x"**).

d. The Pyramid has three layers, and a sphere on the top of the top layer. Therefore, you will be working with 4 y-coordinates: one for the cube(s) in each layer, and one for the sphere at the top.

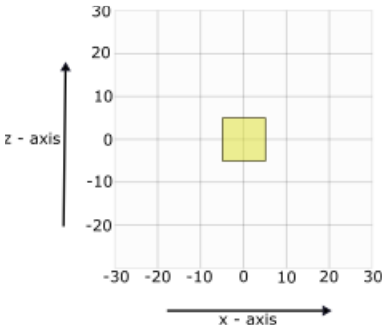
e. The bottom layer is made up of 9 cubes shown in the 2D grid below (showing x and z axes). Therefore, declare 8 **IModel*** variables, create a cube model for each, and then position them using the coordinates provided by the image.



f. The middle layer is made up of 4 cubes shown in the 2D grid below (showing x and z axes). Therefore, declare 4 **IModel*** variables, create a cube model for each, and then position them using the coordinates provided by the image.

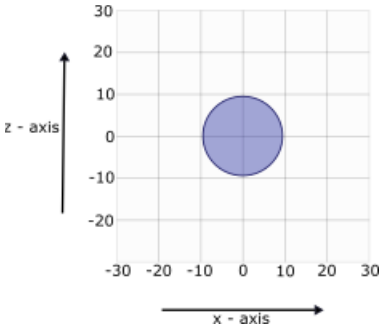
Remember that this layer should be placed on top of the bottom layer; therefore, the y-coordinate for the cubes in this layer should be above the y-coordinate of the cubes in the bottom layer.

g. The top layer is made up of a single cube shown in the 2D grid below (showing x and z axes). Therefore, declare an **IModel*** variable, create a cube model for it, and then position it using the coordinates provided by the image.



Remember that this layer should be placed on top of the middle layer; therefore, the y-coordinate for the cube in this layer should be above the y-coordinate of the cubes in the middle layer.

h. Finally, declare an **IModel*** variable, create a sphere model for it, and then position it using the coordinates provided by the image below:



Remember that this sphere should be placed on top of the top pyramid layer; therefore, the y-coordinate for the sphere should be above the y-coordinate of the cube in the top layer.

i. For a final touch, add code to rotate the sphere slowly along its x-axis.

8. Advanced Movement

The methods below were introduced in the lab; look carefully at the example usage and try writing code to implement the tasks that follow.

```
// The Set functions allow you to set the X, Y, or Z coordinates of a model to a given value
void SetX( float ); // e.g. cube1->SetX( -20.0 );
void SetY( float ); // e.g. cube2->SetY( 10.0 );
void SetZ( float ); // e.g. cube2->SetZ( 7.5 );

// Set all axes at once (as CreateModel)
void SetPosition (float X, float Y, float Z); //e.g.myModel->SetPosition(20, 0, 10);

// The Get functions return the current X, Y or Z position of a model
float GetX(); // e.g. xpos = sphere->GetX();
float GetY(); // e.g. if (sphere->GetY() > 10)
float GetZ(); // e.g. zpos = sphere->GetZ();
```

a. Create a new TL-Engine project called **Lab1B_ControlledSphere_Project**.
b. Load a sphere mesh, then create a sphere model and move it upwards until its Y reaches 30, then return it to its starting position.
c. Update your code so that the sphere bounces left and right between X = -40 and X = 40.
d. Can you make the sphere roll while it bounces?

You will need to use your programming skills (specifically selection using "if" statements).